

CHƯƠNG 6

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Mục đích của chương: Tổng hợp những kết quả chính của quá trình nghiên cứu, thiết kế, triển khai và kiểm thử MyOS; đồng thời chỉ ra giới hạn hiện tại và đề xuất lộ trình phát triển hệ thống theo hướng hoàn thiện, định lượng được và có khả năng ứng dụng trên phần cứng thực.

6.1. Kết luận

Đồ án đã thực hiện quá trình nghiên cứu và xây dựng một hệ điều hành thời gian thực nhúng có tên MyOS trên nền tảng ARM Cortex-M4, hướng đến vi điều khiển STM32F407. Khác với cách chỉ sử dụng một RTOS có sẵn ở mức API, đề tài tiếp cận từ bên trong kernel: tổ chức trạng thái task, xây dựng ngăn xếp ban đầu, thiết kế thuật toán lập lịch, thực hiện chuyển ngữ cảnh bằng Assembly, xử lý tick hệ thống, quản lý danh sách chờ, đồng bộ hóa, giao tiếp liên tiến trình và giám sát tài nguyên. Cách tiếp cận này giúp làm rõ mối liên hệ giữa kiến trúc phần cứng Cortex-M4 với hành vi của một kernel thời gian thực.

Kết quả quan trọng nhất của đề tài không chỉ là một tập hợp các hàm riêng lẻ, mà là một hệ thống phần mềm có cấu trúc tương đối hoàn chỉnh. Mã nguồn được phân tách thành các lớp kernel, architecture port, board, platform, driver, HAL và application. Luồng khởi động đi từ khởi tạo bo mạch, khởi tạo kernel, khởi tạo các dịch vụ dùng chung, tạo task ứng dụng và cuối cùng chuyển quyền điều khiển cho scheduler. Cấu trúc này tạo ranh giới rõ giữa phần phụ thuộc kiến trúc và phần logic kernel, qua đó hỗ trợ việc bảo trì và mở rộng về sau.

6.1.1. Mức độ hoàn thành mục tiêu đề tài

So với mục tiêu đặt ra, MyOS đã hình thành được các thành phần cốt lõi cần thiết của một RTOS nhỏ. Hệ thống có mô hình TCB để lưu ngữ cảnh và thông tin quản lý task; có các trạng thái UNUSED, NEW, READY, RUNNING, BLOCKED,

SUSPENDED và TERMINATED; hỗ trợ tạo task tĩnh hoặc động, delay, yield, suspend, resume, kill và task exit. Các thao tác chuyển trạng thái được gắn với ready list, wait list và cơ chế timeout, giúp vòng đời task được quản lý thống nhất hơn thay vì xử lý phân tán trong từng dịch vụ.

Scheduler của MyOS được thiết kế theo ưu tiên, cho phép preemption và time slicing. Cấu hình hiện tại cho phép tối đa 8 task và 8 mức ưu tiên, sử dụng lát thời gian một tick. Với các task cùng mức ưu tiên, cơ chế round-robin tạo cơ hội sử dụng CPU luân phiên; với các task khác mức ưu tiên, task sẵn sàng có ưu tiên cao hơn có thể yêu cầu lập lịch lại. PendSV được dùng làm điểm thực hiện chuyển ngữ cảnh, trong khi SysTick cung cấp nhịp thời gian cho delay, timeout, software timer và time slicing. Đây là lựa chọn phù hợp với khuyến nghị thiết kế kernel trên Cortex-M vì cho phép dời thao tác chuyển ngữ cảnh xuống exception có ưu tiên thấp.

Nhóm chức năng đồng bộ và IPC đã vượt ra ngoài mức tối thiểu của một kernel minh họa. MyOS có binary semaphore, counting semaphore, mutex, recursive mutex, timeout cho thao tác chờ, priority inheritance, message queue, message buffer và event group. Một số thao tác được cung cấp biến thể dùng trong ISR, chẳng hạn phát semaphore, gửi hoặc nhận queue và đặt bit sự kiện. Việc bổ sung timeout và danh sách chờ giúp API có ý nghĩa thực tế hơn, vì task có thể giới hạn thời gian chờ thay vì bị khóa vô thời hạn.

Đối với quản lý tài nguyên, kernel đã có heap động dung lượng cấu hình 32 KiB, theo dõi dung lượng còn trống, mức thấp nhất từng ghi nhận, block trống lớn nhất, số block trống và tỷ lệ phân mảnh. Stack của task được khởi tạo với mẫu dữ liệu, có vùng guard và stack canary để phát hiện ghi tràn. TCB còn lưu thông tin vùng stack và cấu hình MPU của từng task. Cơ chế chuyển task có thể cập nhật vùng MPU động dành cho stack và một vùng mở rộng, tạo nền tảng cho việc cô lập bộ nhớ theo task.

Bên cạnh các chức năng vận hành, MyOS đã tích hợp nhóm công cụ quan sát gồm trace buffer, runtime statistics, thống kê stack, thống kê heap và UART shell. Người dùng có thể xem danh sách task, mức sử dụng CPU, sự kiện trace, trạng thái heap, queue, năng lượng và stack; đồng thời có thể dừng, tiếp tục hoặc kết thúc một task từ giao diện lệnh. Những công cụ này có giá trị lớn trong giai đoạn phát triển

RTOS vì lỗi thường phụ thuộc thời điểm, trạng thái và thứ tự lập lịch, khó quan sát chỉ bằng breakpoint truyền thống.

STT	Nhóm chức năng	Mức độ	Kết quả chính
1	Quản lý task	Đã triển khai	Tạo tĩnh/động, delay, yield, suspend, resume, kill, exit, truy vấn thông tin task.
2	Lập lịch và chuyển ngữ cảnh	Đã triển khai	Ưu tiên, preemption, time slicing, round-robin, SysTick và PendSV.
3	Đồng bộ hóa	Đã triển khai	Binary/counting semaphore, mutex đệ quy, timeout và priority inheritance.
4	IPC	Đã triển khai	Message queue, message buffer, thao tác timeout và một số API từ ISR.
5	Event group và timer	Đã triển khai	Chờ một/nhiều bit, clear-on-exit; timer one-shot và periodic.
6	Bộ nhớ và an toàn	Đã triển khai ở mức nền tảng	Heap, thống kê phân mảnh, stack canary, guard words và cấu hình MPU theo task.
7	Tiết kiệm năng lượng	Đã triển khai ở mức kernel	Tickless idle, dừng SysTick có điều kiện và bù số tick sau khi thức dậy.
8	Quan sát hệ thống	Đã triển khai	Trace, runtime statistics, UART shell, thông tin task/heap/stack.
9	Ứng dụng minh họa	Đã triển khai	Luồng cảm biến nhiệt độ ảo, lọc trung bình, queue, điều khiển cảnh báo/quạt và

STT	Nhóm chức năng	Mức độ	Kết quả chính
			LED.

6.1.2. Kết quả kiểm thử và bằng chứng vận hành

Đề tài đã xây dựng bộ kịch bản kiểm thử chạy theo tham số biên dịch MYOS_TEST_SCENARIO. Các kịch bản bao phủ delay và timeout, timeout semaphore, suspend task đang delay, kết thúc task đang chờ, round-robin, priority inheritance, phân mảnh heap, phát hiện tràn stack, quyền sở hữu mutex, timeout queue, xử lý mutex khi task bị kill hoặc suspend, thao tác semaphore từ ISR, binary semaphore, counting semaphore, recursive mutex, event group và software timer. Mỗi kịch bản phát kết quả qua UART theo định dạng thống nhất, nhờ đó script PowerShell có thể biên dịch từng cấu hình, chạy trên Renode hoặc QEMU, thu log và xác định PASS/FAIL theo mẫu.

Trong các log được lưu cùng dự án, kết quả PASS đã được ghi nhận cho kịch bản round-robin, mutex priority inheritance và event group. Riêng log round-robin gần nhất cho thấy hai task đồng mức ưu tiên đều thực hiện đủ 8 lượt và có 15 lần chuyển đổi quan sát được, phù hợp với mục tiêu kiểm tra luân phiên. Các log ứng dụng trước đó cũng cho thấy luồng xử lý nhiệt độ có thể chuyển qua các vùng NORMAL, WARN và CRITICAL, thay đổi mức điều khiển quạt và phát thông báo cảnh báo. Tuy nhiên, để bảo đảm tính khoa học, chương này không suy rộng các bằng chứng trên thành kết luận rằng toàn bộ kịch bản đã được xác nhận trên mọi phiên bản mã nguồn và mọi phần cứng.

Phạm vi khẳng định: Mã nguồn hiện có chứa đầy đủ các kịch bản kiểm thử nêu trên, nhưng bằng chứng log lưu trữ chỉ xác nhận trực tiếp một số kịch bản tiêu biểu. Việc nghiệm thu cuối cùng cần chạy lại toàn bộ test suite trên cùng commit phát hành, lưu phiên bản toolchain, cấu hình board và log kết quả để bảo đảm khả năng tái lập.

6.1.3. Giá trị kỹ thuật và học thuật

Về mặt kỹ thuật, MyOS cho thấy một kernel RTOS nhỏ có thể được xây dựng theo hướng phân lớp và có công cụ quan sát ngay từ đầu. Việc tách phần port Cortex-M4 khỏi scheduler, task manager và dịch vụ đồng bộ giúp giảm phụ thuộc phần cứng trong logic kernel. Các cấu trúc danh sách intrusive được dùng chung cho ready list, blocked list và wait list giúp hạn chế cấp phát động cho node quản lý. Cơ chế critical section, cập nhật trạng thái task và yêu cầu PendSV tạo nên chuỗi xử lý có thể theo dõi được từ API mức cao xuống kiến trúc CPU.

Về mặt học thuật, quá trình triển khai làm rõ các khái niệm vốn dễ bị xem như hộp đen khi chỉ sử dụng RTOS thương mại: khung stack ban đầu của task, sự khác biệt giữa MSP và PSP, cơ chế tự động lưu một phần thanh ghi khi exception xảy ra, vai trò của PendSV, quan hệ giữa độ ưu tiên ngắt và tính đúng đắn của API FromISR, nguyên nhân của priority inversion, cách timeout làm thay đổi trạng thái task và tác động của phân mảnh heap đến khả năng cấp phát. Đây là giá trị nổi bật của đề tài đối với việc đào tạo hệ điều hành, kiến trúc máy tính và hệ thống nhúng.

Ứng dụng điều khiển nhiệt độ minh họa cũng chứng minh khả năng kết hợp nhiều dịch vụ kernel trong một luồng xử lý: task cảm biến tạo dữ liệu và lọc trung bình; message queue truyền dữ liệu sang task điều khiển; mutex bảo vệ trạng thái dùng chung và UART; task điều khiển phân loại vùng nhiệt độ, quyết định mức quạt và phát cảnh báo; task shell cung cấp khả năng quan sát; task GPIO biểu diễn hoạt động nền. Dù cảm biến hiện là mô hình ảo, kiến trúc ứng dụng đã thể hiện cách một bài toán nhúng được phân rã thành các task có trách nhiệm rõ ràng.

6.1.4. Những hạn chế còn tồn tại

Mặc dù đạt được mục tiêu nghiên cứu, MyOS hiện vẫn phù hợp nhất với vai trò kernel học tập và nguyên mẫu nghiên cứu, chưa nên được xem là RTOS sẵn sàng cho sản phẩm thương mại hoặc hệ thống an toàn cao. Giới hạn đầu tiên là mức độ kiểm chứng. Bộ test hiện tập trung vào kiểm thử kịch bản trên mô phỏng, chưa có báo cáo coverage, stress test dài hạn, fuzzing, kiểm thử race condition có hệ thống hoặc ma trận kiểm thử trên nhiều mức tối ưu hóa compiler. Một số lỗi thời gian thực chỉ xuất

hiện khi tải ngắt cao, khi tick tràn số, khi nhiều timeout hết hạn đồng thời hoặc khi hệ thống chạy liên tục trong thời gian dài.

Giới hạn thứ hai thuộc về tính xác định và hiệu năng. Scheduler dùng bitmap là lựa chọn tốt cho số mức ưu tiên nhỏ, nhưng cần đo định lượng thời gian scheduler, context switch, thời gian khóa critical section và độ trễ đánh thức task. Software timer hiện cần được đánh giá kỹ về thời gian thực thi callback; nếu callback chạy trực tiếp trong ngữ cảnh xử lý tick hoặc kernel thì một callback dài có thể làm tăng jitter và trì hoãn dịch vụ khác. Heap động có thống kê phân mảnh nhưng chưa chứng minh được giới hạn thời gian cấp phát/giải phóng trong trường hợp xấu nhất.

Giới hạn thứ ba là bảo vệ hệ thống. MPU đã có cơ sở cấu hình vùng tĩnh và vùng động theo task, nhưng mô hình đặc quyền, cổng system call, quyền truy cập ngoại vi, kiểm tra con trỏ người dùng và xử lý MemManage fault cần được hoàn thiện thành một chuỗi bảo vệ nhất quán. Nếu task vẫn chạy đặc quyền hoặc có thể gọi trực tiếp mọi API nội bộ, MPU mới chủ yếu hỗ trợ phát hiện lỗi chứ chưa tạo được ranh giới bảo mật giống mô hình user/kernel.

Giới hạn thứ tư là phạm vi phần cứng và middleware. Driver hiện tập trung vào UART và GPIO; ứng dụng minh họa sử dụng cảm biến nhiệt độ ảo. Hệ thống chưa có lớp driver đầy đủ cho timer phần cứng, DMA, I2C, SPI, ADC, RTC, watchdog và lưu trữ ngoài; chưa có file system, TCP/IP stack, cơ chế cập nhật firmware hoặc quản lý thiết bị chuẩn hóa. Do đó khả năng ứng dụng vào bài toán thực tế còn phụ thuộc đáng kể vào việc mở rộng tầng HAL/driver.

6.1.5. Kết luận chung

Từ các kết quả đã trình bày, có thể kết luận rằng đề tài đã hoàn thành mục tiêu xây dựng một RTOS nhúng cơ bản trên ARM Cortex-M4 và tạo được nền tảng thực nghiệm có giá trị. MyOS đã vận hành được chuỗi chức năng từ khởi động phần cứng, tạo task, lập lịch, chuyển ngữ cảnh, đồng bộ và IPC đến quản lý bộ nhớ, giám sát và ứng dụng minh họa. Thiết kế hiện tại thể hiện sự liên kết hợp lý giữa lý thuyết và triển khai, đồng thời đủ rõ ràng để tiếp tục nghiên cứu các chủ đề nâng cao như bảo vệ bộ nhớ, kiểm chứng thời gian thực, tiết kiệm năng lượng và middleware nhúng.

Dù chưa đạt mức độ hoàn thiện của các RTOS phổ biến như FreeRTOS, giá trị của MyOS nằm ở khả năng quan sát và kiểm soát toàn bộ đường đi của kernel. Người phát triển có thể trực tiếp thay đổi thuật toán lập lịch, cấu trúc TCB, chính sách cấp phát, cơ chế trace hoặc cách xử lý exception để đánh giá ảnh hưởng. Vì vậy, MyOS là một nền tảng phù hợp cho học tập, thực hành và thử nghiệm ý tưởng hệ điều hành trên vi điều khiển.

6.2. Hướng phát triển

Hướng phát triển của MyOS cần được sắp xếp theo thứ tự ưu tiên thay vì mở rộng đồng thời tất cả chức năng. Trước hết cần củng cố tính đúng đắn, khả năng tái lập và an toàn của kernel; tiếp theo mới tối ưu hiệu năng, mở rộng driver và middleware; cuối cùng là chuẩn hóa tài liệu và quy trình phát hành. Lộ trình như vậy giúp tránh tình trạng số lượng API tăng nhanh nhưng nền tảng kiểm thử và chẩn đoán không theo kịp.

6.2.1. Hoàn thiện MPU và mô hình đặc quyền

Ưu tiên đầu tiên là hoàn thiện MPU thành cơ chế bảo vệ thực sự. Kernel cần chạy ở privileged mode, trong khi task ứng dụng thông thường chạy ở unprivileged Thread mode và sử dụng PSP. Các dịch vụ nhạy cảm như tạo task, cấu hình MPU, truy cập NVIC, quản lý heap kernel hoặc điều khiển thiết bị phải đi qua SVC. SVC handler cần giải mã số system call, kiểm tra tham số, xác thực địa chỉ và chỉ thực hiện thao tác được cho phép.

Mỗi task nên có vùng stack riêng với quyền đọc/ghi và XN, vùng code chỉ đọc/thực thi, vùng dữ liệu theo nhu cầu và tối đa một số vùng ngoại vi được cấp quyền. Khi task chuyển đổi, kernel cập nhật các slot MPU động trước khi trả về Thread mode. MemManage handler cần thu thập CFSR, MMFAR, PC, LR và TID gây lỗi, ghi trace và kết thúc task vi phạm hoặc chuyển hệ thống sang trạng thái an toàn. Cần bổ sung kiểm thử âm: task ghi vào stack task khác, ghi vào flash, thực thi từ RAM XN, truy cập ngoại vi không được cấp quyền và truyền con trỏ ngoài vùng hợp lệ cho system call.

6.2.2. Mở rộng và chuẩn hóa driver

Tầng driver nên được phát triển theo giao diện thống nhất gồm khởi tạo, cấu hình, truyền nhận đồng bộ, truyền nhận bất đồng bộ, timeout và callback từ ISR. Các driver ưu tiên gồm UART có ring buffer, I2C, SPI, ADC, timer, PWM, DMA, RTC và independent watchdog. Driver không nên gọi trực tiếp logic ứng dụng trong ISR; ISR chỉ xác nhận nguồn ngắt, di chuyển dữ liệu tối thiểu và đánh thức task xử lý thông qua semaphore, queue hoặc event group.

Sau khi có I2C/SPI/ADC, ứng dụng nhiệt độ nên thay cảm biến ảo bằng cảm biến thật, chẳng hạn cảm biến số qua I2C hoặc kênh ADC. PWM phần cứng được dùng để điều khiển quạt, GPIO điều khiển LED cảnh báo và watchdog phát hiện kernel hoặc task bị treo. Việc chạy trên board thật cần đo bằng logic analyzer hoặc oscilloscope để xác nhận chu kỳ lấy mẫu, jitter, độ trễ từ ngắt đến task và độ chính xác của tickless idle.

6.2.3. Bổ sung file system và lưu trữ

File system là bước cần thiết để MyOS hỗ trợ ghi log, cấu hình và dữ liệu cảm biến. Có thể tích hợp FatFs trên thẻ SD hoặc xây dựng một lớp lưu trữ nhỏ cho SPI flash. Phần port phải cung cấp primitive khóa, thời gian và driver block device. Các thao tác I/O chậm nên được thực hiện bởi một task lưu trữ chuyên trách; task khác gửi yêu cầu qua queue để tránh giữ mutex lâu và giảm ảnh hưởng đến deadline.

Cần xem xét tình huống mất điện trong lúc ghi, giới hạn chu kỳ ghi flash, tính toàn vẹn metadata và cơ chế phục hồi sau reset. Với cấu hình nhỏ, có thể dùng log dạng append-only, CRC cho từng record và hai vùng cấu hình luân phiên. Khi sử dụng FatFs, cần kiểm thử mount/unmount, thẻ đầy, lỗi CRC, rút thẻ đột ngột và truy cập đồng thời.

6.2.4. Tích hợp TCP/IP và kết nối mạng

Để mở rộng sang IoT, MyOS có thể tích hợp một TCP/IP stack gọn như lwIP. Công việc này đòi hỏi driver Ethernet hoặc Wi-Fi, bộ đệm packet, timer giao thức, cơ

chế đồng bộ với ISR và chiến lược cấp phát bộ nhớ phù hợp. Giai đoạn đầu nên hỗ trợ DHCP, ARP, ICMP và UDP; sau đó mới bổ sung TCP, DNS, MQTT hoặc HTTP.

Thiết kế mạng phải kiểm soát áp lực bộ nhớ và hiện tượng priority inversion giữa task mạng với task thời gian thực. Pool buffer kích thước cố định thường phù hợp hơn heap tổng quát cho packet. Cần đặt giới hạn queue, timeout và chính sách loại bỏ packet khi quá tải. Với giao tiếp ra ngoài, bảo mật trở thành yêu cầu bắt buộc: xác thực thiết bị, bảo vệ khóa, kiểm tra dữ liệu đầu vào và cập nhật firmware có chữ ký.

6.2.5. Benchmark và đánh giá định lượng

Một hướng phát triển quan trọng là chuyển từ đánh giá định tính sang số liệu định lượng. Bộ benchmark cần đo thời gian context switch, thời gian lập lịch khi số task và mức ưu tiên thay đổi, độ trễ semaphore/queue, thời gian đánh thức task từ ISR, jitter của task định kỳ, chi phí critical section, thời gian cấp phát/giải phóng heap và chi phí cấu hình MPU trong mỗi lần chuyển task. Kết quả nên báo cáo giá trị nhỏ nhất, trung bình, phân vị cao và trường hợp xấu nhất quan sát được.

Có thể dùng DWT cycle counter của Cortex-M4 để đo số chu kỳ CPU, kết hợp GPIO toggle và logic analyzer để kiểm tra độc lập. Benchmark phải chạy ở các mức tối ưu hóa khác nhau, có và không có FPU, có và không có trace, trên mô phỏng và board thật. Khi so sánh với FreeRTOS, cần giữ nguyên xung nhịp, compiler, mức tối ưu hóa, kích thước stack, tải task và cách đo để tránh kết luận thiên lệch.

6.2.6. Tự động hóa kiểm thử và tích hợp liên tục

Bộ script kiểm thử hiện có nên được đưa vào quy trình CI. Mỗi commit cần biên dịch cấu hình chuẩn, cấu hình test và cấu hình bật/tắt từng tính năng; sau đó chạy test trên QEMU hoặc Renode, thu log và xuất báo cáo máy đọc được. Test suite cần được mở rộng cho tick wrap-around, nhiều timeout đồng thời, queue nhiều producer/consumer, nested interrupt, task bị kill khi đang giữ nhiều mutex, callback timer tự dừng/reset, cấp phát ngẫu nhiên và chuyển task khi dùng thanh ghi FPU.

Ngoài test tích hợp, các cấu trúc dữ liệu độc lập như list, heap, bitmap scheduler và hàm encode MPU có thể được unit test trên host. Static analysis với compiler

warning nghiêm ngặt, clang-tidy hoặc cppcheck giúp phát hiện truy cập NULL, chuyển kiểu nguy hiểm và nhánh không thể tới. Sanitizer có thể áp dụng cho phiên bản host của một số module. Mỗi lần phát hành cần lưu commit hash, phiên bản GCC, linker map, kích thước firmware và toàn bộ log PASS/FAIL.

6.2.7. Nâng cao độ tin cậy và khả năng chẩn đoán

Kernel panic và fault handler nên được phát triển thành hệ thống crash dump. Khi HardFault, MemManage, BusFault hoặc UsageFault xảy ra, hệ thống ghi thanh ghi xếp chồng, MSP, PSP, CFSR, HFSR, MMFAR, BFAR, TID hiện hành, watermark stack và các sự kiện trace cuối cùng vào vùng RAM giữ lại hoặc flash. Sau reset, UART shell có thể in bản ghi để hỗ trợ phân tích.

Các assert nội bộ cần kiểm tra tính nhất quán của danh sách, trạng thái task, chủ sở hữu mutex, số lượng semaphore, biên queue và vùng heap. Watchdog nên được phục vụ bởi một task giám sát chỉ khi các task quan trọng đã cập nhật heartbeat. Có thể bổ sung deadline monitor để phát hiện task định kỳ chạy muộn, cùng với cơ chế health status cho toàn hệ thống.

6.2.8. Tối ưu quản lý thời gian và năng lượng

Tickless idle cần được đánh giá và hoàn thiện cho các tình huống biên như ngắt xảy ra ngay trước khi WFI, thời gian ngủ vượt khả năng đếm của SysTick, nhiều nguồn đánh thức và sai số do xung nhịp. Về lâu dài, có thể dùng timer 32 bit hoặc RTC làm nguồn thời gian khi ngủ sâu. Kernel nên duy trì monotonic time độc lập với số tick đã bỏ qua và cung cấp API chuyển đổi giữa tick với microsecond/millisecond an toàn trước tràn số.

Chính sách năng lượng có thể mở rộng thành nhiều trạng thái: idle bằng WFI, sleep, stop và standby. Mỗi driver cần có hook prepare/resume để tắt và khôi phục ngoại vi. Bộ quản lý năng lượng chỉ chọn trạng thái ngủ sâu khi deadline gần nhất, trạng thái DMA, UART và yêu cầu của ứng dụng cho phép. Hiệu quả cần được đo bằng dòng tiêu thụ thực tế thay vì chỉ dựa trên số tick được bỏ qua.

6.2.9. Chuẩn hóa API, tài liệu và khả năng chuyển nền tảng

API cần được chuẩn hóa về tiền tố, kiểu trả về, quy tắc timeout và hành vi khi gọi từ ISR. Mỗi API nên mô tả rõ tham số, ngữ cảnh được phép gọi, khả năng block, độ phức tạp, lỗi trả về và yêu cầu đồng bộ. Các cấu hình kernel nên được kiểm tra bằng compile-time assertion; những tính năng tùy chọn phải có dependency rõ để tránh cấu hình hợp lệ về cú pháp nhưng không nhất quán về chức năng.

Để tăng khả năng chuyển nền tảng, phần phụ thuộc Cortex-M4 cần được gom vào port layer với giao diện ổn định. Sau đó có thể thử port sang Cortex-M3 hoặc Cortex-M7, hoặc sang một board STM32 khác. Việc port thành công sẽ là phép kiểm tra tốt cho mức độ phân lớp của kiến trúc. Tài liệu nên bổ sung sơ đồ trạng thái task, sequence diagram cho context switch, mô tả stack frame, quy tắc priority, hướng dẫn viết driver và quy trình thêm một board mới.

6.2.10. Lộ trình triển khai đề xuất

Giai đoạn	Mục tiêu	Công việc trọng tâm	Đầu ra kỳ vọng
Giai đoạn 1	Ổn định nền tảng	Chạy lại toàn bộ test; CI; crash dump; kiểm tra tick wrap-around; benchmark cơ sở.	Bản phát hành có log tái lập và bộ chỉ số chuẩn.
Giai đoạn 2	An toàn bộ nhớ	Task unprivileged; SVC syscall; xác thực con trỏ; MPU test âm; MemManage recovery.	Cô lập lỗi giữa task và kernel.
Giai đoạn 3	Phần cứng thực	I2C/SPI/ADC/PWM/DMA/watchdog; cảm biến thật; đo jitter và latency.	Ứng dụng nhiệt độ vận hành ổn định trên STM32F407.
Giai đoạn 4	Middleware	File system, lưu log, RTC; sau đó lwIP và giao thức ứng dụng.	Nền tảng ứng dụng nhúng có lưu trữ và kết nối.
Giai đoạn 5	Hoàn thiện sản phẩm nghiên cứu	Tài liệu API, coding standard, release package, so sánh	MyOS trở thành nền tảng giảng dạy và thực nghiệm có thể tái sử

Giai đoạn	Mục tiêu	Công việc trọng tâm	Đầu ra kỳ vọng
		FreeRTOS công bằng.	dụng.

6.2.11. Định hướng kết thúc

Định hướng dài hạn phù hợp nhất của MyOS là trở thành một RTOS nghiên cứu nhỏ, minh bạch và có khả năng đo lường, thay vì cố gắng sao chép toàn bộ chức năng của các hệ điều hành đã trưởng thành. Những tính năng được bổ sung cần phục vụ ba mục tiêu: làm rõ nguyên lý hệ điều hành, tạo môi trường kiểm chứng trên Cortex-M và hỗ trợ xây dựng các ứng dụng nhúng quy mô nhỏ. Mỗi tính năng mới phải đi kèm test, số liệu và tài liệu thiết kế.

Nếu thực hiện theo lộ trình trên, MyOS có thể tiến từ một kernel đồ án sang một nền tảng thực nghiệm đáng tin cậy hơn: có ranh giới bảo vệ bộ nhớ, có driver cho phần cứng thật, có kiểm thử tự động, có benchmark định lượng và có middleware cơ bản. Quan trọng hơn, quá trình phát triển tiếp theo sẽ tiếp tục duy trì giá trị cốt lõi của đề tài: giúp người học quan sát được cách một hệ điều hành thời gian thực hoạt động từ thanh ghi CPU đến API ứng dụng.